24 EXCEPTION HANDLING — HOW A throw BECOMES AN HTTP RESPONSE

This project never manually writes an error response (no res.status(404).json(...) anywhere). Every "not found" / "unauthorized" / "forbidden" / "bad request" in the codebase is just a JavaScript throw:

```
throw new NotFoundException(`Task ${id} not found`);   // task.service.ts
throw new UnauthorizedException('Invalid or missing API key'); // auth.guard.ts
throw new ForbiddenException(`Requires role(s): ...`);  // roles.guard.ts
throw new BadRequestException(`${metadata.data} must be an integer...`); // parse-positive-int.pipe.ts
```

None of these files touch the HTTP response directly. Nest catches the throw for you, wherever it happens, and turns it into a JSON error body with the right status code. That automatic catch-and-convert step is "exception handling" in Nest.

THE BUILT-IN GLOBAL EXCEPTION FILTER

Every Nest app has one of these running from the moment it starts, with zero setup — it's baked into NestFactory.create() in main.ts. It sits

OUTSIDE the normal request pipeline, wrapping the whole thing:

```
request -> Guards -> Pipes -> Controller -> Service
              |         |          |            |
              +---------+----------+------------+
                        |
                   any throw() here
                        |
                        v
              built-in ExceptionsHandler
                        |
                        v
                  JSON error response
```

It doesn't matter WHERE in that chain the throw happens — a guard (auth.guard.ts), a pipe (parse-positive-int.pipe.ts), or a service (task.service.ts, user.service.ts). All four throw sites in this project end up funneled through the exact same catch-all. That's the whole point: none of those files need a try/catch, because there is nowhere for the exception to "leak" to — Nest is always listening one level up.

HttpException — THE BASE CLASS EVERYTHING HERE EXTENDS

NotFoundException, UnauthorizedException, ForbiddenException, and BadRequestException are not four unrelated classes — they're thin presets over one base class, HttpException, which takes two things:

```
new HttpException(response, status)
                      |         |
                      |         +-- the HTTP status code to send
                      +-- the response BODY (string or object)

new NotFoundException(`Task ${id} not found`)
```

```
  is shorthand for
new HttpException(`Task ${id} not found`, 404)
```

Each built-in subclass just hardcodes the status:

BadRequestException	→ 400	(parse-positive-int.pipe.ts)
UnauthorizedException	→ 401	(auth.guard.ts)
ForbiddenException	→ 403	(roles.guard.ts)
NotFoundException	→ 404	(task.service.ts, user.service.ts)

Picking the right subclass IS the design decision — auth.guard.ts uses 401 (identity unproven: bad/missing API key) while roles.guard.ts uses 403 (identity proven, just not permitted). Same shape of check, two different codes, because they mean different things to a client.

WHAT THE CLIENT ACTUALLY RECEIVES

The built-in filter turns the exception into a fixed JSON shape:

```
throw new NotFoundException(`Task 99 not found`);

HTTP/1.1 404 Not Found
{
  "statusCode": 404,
  "message": "Task 99 not found",
  "error": "Not Found"
}
```

The string passed to the constructor becomes "message". If you instead pass an object, that object becomes the body verbatim (statusCode gets merged in) — useful for structured error payloads, though nothing in this project needs that yet.

VALIDATIONPIPE: EXCEPTIONS YOU NEVER WRITE YOURSELF

main.ts registers a global pipe:

```
app.useGlobalPipes(
  new ValidationPipe({
    whitelist: true,           // strip unknown DTO fields
    forbidNonWhitelisted: true, // reject the request if any were sent
  }),
);
```

Nowhere does this project throw a validation error by hand — Validation Pipe does it internally, and it's still a BadRequestException under the hood, still caught by the exact same built-in filter. forbidNonWhite listed: true is what turns "client sent an extra field" into a 400 instead of silently dropping it. See theory/09-pipes.txt and theory/15-class-validator-and-class-transformer.txt for how the DTO rules that trigger this are declared.

UNCAUGHT, NON-HttpException ERRORS → 500

throw new NotFoundException(...) is a DELIBERATE, recognized exception → Nest reads its status and message and reports them honestly. But a genuine bug — say task.service.ts had a typo like this.stasks.push(...) — throws a plain TypeError, which is NOT an HttpException. The same built-in filter still catches it (nothing ever crashes the process), but since it doesn't know what status that represents, it reports the safe default:

```
HTTP/1.1 500 Internal Server Error
{ "statusCode": 500, "message": "Internal server error" }
```

and logs the real error server-side. This is why services in this project throw NotFoundException instead of returning null or a plain Error — it's the difference between the client seeing an accurate 404 versus an opaque 500 for what was really just "not found".

CUSTOM EXCEPTION FILTERS — THE NEXT STEP THIS PROJECT DOESN'T NEED YET

Every filter used so far is the built-in default. Nest lets you REPLACE or ADD to it with @Catch() + implements ExceptionFilter, the same shape as a pipe (theory/10-custom-pipes.txt) or a guard (theory/17-guards.txt) — a Nest-defined interface you implement, then attach with a decorator:

```
@Catch(HttpException)
export class HttpExceptionFilter implements ExceptionFilter {
  catch(exception: HttpException, host: ArgumentsHost) {
    const ctx = host.switchToHttp();
    const response = ctx.getResponse<Response>();
    const status = exception.getStatus();

    response.status(status).json({
      statusCode: status,
      timestamp: new Date().toISOString(),
      path: ctx.getRequest<Request>().url,
      message: exception.getResponse(),
    });
  }
}
```

@Catch(HttpException) scopes it to only HttpException (and its subclasses) — a bare @Catch() with no argument catches everything, including the plain TypeError case above, which is how you'd add consistent logging for truly unexpected errors too.

Attaching it mirrors guards/pipes exactly — method, controller, or global scope:

```
@UseFilters(HttpExceptionFilter)   // one route or one controller
app.useGlobalFilters(new HttpExceptionFilter()); // every route, in main.ts
```

This project has no reason to add one — the default JSON shape (statusCode/message/error) is already enough for every route here. The trigger to actually add one is wanting a RESPONSE SHAPE the built-in filter doesn't produce (a `path` and `timestamp` field, a company-wide error envelope, structured logging per exception) — not a requirement for exceptions to work at all.

WHERE THIS FITS IN THE REQUEST LIFECYCLE

Exception filters are the ONE stage that runs outside the normal guard → pipe → handler order, because they can fire FROM any of the others:

```
Middleware -> Guards -> Interceptors(before) -> Pipes -> Handler -> Service
     |           |              |                 |         |         |
     +-----------+--------------+-----------------+---------+---------+
                                |
                          throw ANYWHERE above
                                |
                                v
                       Exception Filter (built-in or custom)
                                |
                                v
                          HTTP response sent
```

roles.guard.ts throwing ForbiddenException and task.service.ts throwing NotFoundException hit the identical filter stage despite firing from completely different layers — filters don't care where the exception came from, only what TYPE it is.

★ RULE OF THUMB

Throw the built-in HttpException subclass that matches what actually happened (NotFoundException for "doesn't exist", UnauthorizedException for "who are you", ForbiddenException for "I know who you are, and no", BadRequestException for "malformed input") and let Nest's default filter turn it into JSON — never hand-build a response object yourself. Reach for a custom @Catch() filter only when the DEFAULT response shape itself is the problem, not as a substitute for picking the right exception class in the first place.

See also: theory/09-pipes.txt, theory/10-custom-pipes.txt, theory/17-guards.txt, theory/22-role-based-authorization.txt, theory/23-reflector-and-metadata.txt